

Roteiro de Casos de Testes — TransparênciaPE Backend

Projeto: TransparênciaPE — API de Dados Fiscais do Estado de Pernambuco

Disciplina: Verificação e Validação de Software

Tipo de Teste: Testes Unitários

Framework: xUnit + Moq + FluentAssertions

Cobertura Alvo: Camadas Application, Domain, Infrastructure (ExternalClients), API (Controllers, Middlewares)

Data: 10/05/2026

Sumário

Casos de Teste (CTs) correspondem a métodos de teste — um cenário de comportamento por método.

Execuções xUnit é o número real reportado ao rodar `dotnet test`: métodos `[Theory]` com múltiplos `[InlineData]` são executados uma vez por dado, gerando mais execuções que CTs.

Módulo	CTs (métodos)	Execuções xUnit	Observação
Helpers — CnpjHelper	4	15	4 <code>[Theory]</code> com InlineData
Helpers — McaspMapper	2	14	1 <code>[Theory]</code> com 13 InlineData
Entidades — OrgaoGoverno	5	5	—
Entidades — Receita	1	1	—
Entidades — Orcamento	1	1	—
Services — DashboardService	5	5	—
Services — PesquisaService	7	9	1 <code>[Theory]</code> com 3 InlineData
Services — DataSyncService	8	8	—
Controllers — DashboardController	5	5	—
Controllers — PesquisaController	3	3	—
ExternalClients — TcePEDataClient	7	7	—
Middlewares — GlobalExceptionMiddleware	6	6	—
Total	54	79	+25 via <code>[InlineData]</code>

Módulo: Helpers — CnpjHelper

Classe utilitária estática para sanitização e validação de CNPJs.

CT-001 — Sanitize remove pontuação de CNPJ formatado

Arquivo: `Helpers/CnpjHelperTests.cs`

Método em teste: `CnpjHelper.Sanitize(string?)`

Pré-condições: Nenhuma. Método estático, sem dependências externas.

Dados de Entrada:

Entrada	Saída Esperada
"11.222.333/0001-81"	"11222333000181"
"11222333000181"	"11222333000181"
"00.000.000/0000-00"	"00000000000000"
" 11.222.333/0001-81 "	"11222333000181"

Passos:

1. Chamar `CnpjHelper.Sanitize(input)` para cada valor de entrada acima.
2. Comparar o retorno com o valor esperado.

Resultado Esperado: Todos os caracteres não-numéricos e espaços são removidos; o CNPJ retorna apenas com 14 dígitos numéricos.

Status:  Passou

CT-002 — Sanitize retorna string vazia para entrada nula ou branca

Arquivo: `Helpers/CnpjHelperTests.cs`

Método em teste: `CnpjHelper.Sanitize(string?)`

Pré-condições: Nenhuma.

Dados de Entrada: `null`, `""`, `" "`

Passos:

1. Chamar `CnpjHelper.Sanitize(input)` para cada valor.
2. Verificar que o retorno é `string.Empty`.

Resultado Esperado: Método retorna `""` sem lançar exceção para entradas nulas ou apenas com espaços.

Status:  Passou

CT-003 — IsValid retorna true para CNPJs válidos

Arquivo: `Helpers/CnpjHelperTests.cs`

Método em teste: `CnpjHelper.IsValid(string?)`

Pré-condições: Nenhuma.

Dados de Entrada: "11222333000181", "11.222.333/0001-81"

Passos:

1. Chamar `CnpjHelper.IsValid(cnpj)` para cada valor.
2. Verificar que retorna `true`.

Resultado Esperado: Retorna `true` para CNPJs numericamente válidos (dígitos verificadores corretos).

Status:  Passou

CT-004 — IsValid retorna false para CNPJs inválidos

Arquivo: `Helpers/CnpjHelperTests.cs`

Método em teste: `CnpjHelper.IsValid(string?)`

Pré-condições: Nenhuma.

Dados de Entrada: "000000000000000", "111111111111111", "12345", "", null, "1234567890123456"

Passos:

1. Chamar `CnpjHelper.IsValid(input)` para cada valor.
2. Verificar que retorna `false`.

Resultado Esperado: Retorna `false` para: todos os dígitos iguais, tamanho incorreto, vazio e nulo.

Status:  Passou

Módulo: Helpers — McaspMapper

Classifica naturezas de despesa MCASP em categorias legíveis.

CT-005 — MapToClassificacao retorna a categoria correta por prefixo

Arquivo: `Helpers/McaspMapperTests.cs`

Método em teste: `McaspMapper.MapToClassificacao(string, string)`

Pré-condições: Nenhuma.

Dados de Entrada:

Natureza	Resultado Esperado
"3.1.90.11"	"Pessoal e Encargos Sociais"
"3.1.00.00"	"Pessoal e Encargos Sociais"
"3.3.90.30"	"Custeio"
"3.3.90.39"	"Custeio"
"4.4.90.51"	"Investimentos"

Natureza	Resultado Esperado
"4.4.90.61"	"Investimentos"
"3.2.90.00"	"Outros"
"4.5.90.52"	"Outros"
"4.6.90.71"	"Outros"
"5.0.00.00"	"Outros"
"10.0.00.00"	"Outros"
""	"Outros"
" "	"Outros"

Passos:

1. Chamar `McaspMapper.MapToClassificacao(natureza, "")` para cada entrada.
2. Comparar o resultado com a categoria esperada.

Resultado Esperado: Prefixos 3.1.* → Pessoal, 3.3.* → Custeio, 4.4.* → Investimentos, demais → Outros.

Status:  Passou

CT-006 — MapToClassificacao retorna "Outros" quando natureza é null

Arquivo: `Helpers/McaspMapperTests.cs`

Método em teste: `McaspMapper.MapToClassificacao(string, string)`

Pré-condições: Nenhuma.

Dados de Entrada: `null`

Passos:

1. Chamar `McaspMapper.MapToClassificacao(null!, "")`.
2. Verificar que retorna "Outros".

Resultado Esperado: Tratamento defensivo via `IsNullOrWhiteSpace` evita `NullReferenceException`.

Status:  Passou

Módulo: Entidades — OrgaoGoverno

Entidade de domínio que representa um órgão governamental.

CT-007 — OrgaoGoverno gera Id único a cada instanciação

Arquivo: `Entities/OrgaoGovernoTests.cs`

Pré-condições: Nenhuma.

Passos:

1. Criar duas instâncias de **OrgaoGoverno** separadamente.
2. Comparar os **Id** gerados.

Resultado Esperado: Ambos os **Id** são **Guid** não-vazios e distintos entre si.

Status:  Passou

CT-008 — OrgaoGoverno inicializa coleções de Empenhos e Contratos como vazias

Arquivo: **Entities/OrgaoGovernoTests.cs**

Pré-condições: Nenhuma.

Passos:

1. Instanciar **new OrgaoGoverno()**.
2. Verificar **Empenhos** e **Contratos**.

Resultado Esperado: As coleções são não-nulas e estão vazias. Não ocorre **NullReferenceException** ao acessá-las.

Status:  Passou

CT-009 — OrgaoGoverno define CreatedAt como UTC no momento da criação

Arquivo: **Entities/OrgaoGovernoTests.cs**

Pré-condições: Nenhuma.

Passos:

1. Registrar **antes = DateTime.UtcNow**.
2. Instanciar **new OrgaoGoverno()**.
3. Verificar que **CreatedAt** está entre **antes** e **DateTime.UtcNow** e que **UpdatedAt** é **null**.

Resultado Esperado: **CreatedAt** é populado automaticamente pela **BaseEntity**; **UpdatedAt** começa nulo.

Status:  Passou

CT-010 — OrgaoGoverno aceita atribuição deCodigo, Nome, Sigla e Tipo

Arquivo: **Entities/OrgaoGovernoTests.cs**

Pré-condições: Nenhuma.

Dados de Entrada: **Codigo**="001", **Nome**="Secretaria de Educação", **Sigla**="SEDUC", **Tipo**="Secretaria"

Passos:

1. Criar **OrgaoGoverno** com os valores acima.
2. Verificar cada propriedade.

Resultado Esperado: Todas as propriedades retornam exatamente os valores atribuídos.

Status:  Passou

CT-011 — OrgaoGoverno armazena TotalServidores e OrcamentoAtual

Arquivo: `Entities/OrgaoGovernoTests.cs`

Pré-condições: Nenhuma.

Dados de Entrada: `TotalServidores=1500, OrcamentoAtual=5.000.000`

Passos:

1. Instanciar com os valores.
2. Verificar ambas as propriedades.

Resultado Esperado: Valores numéricos são armazenados sem perda de precisão.

Status:  Passou

Módulo: Entidades — Receita e Orcamento

CT-012 — Receita é instanciada com propriedades corretas

Arquivo: `Entities/ReceitaTests.cs`

Dados de Entrada: `Valor=150000, Mes=1, Ano=2026, Origem="Impostos", OrgaoGovernoId=<Guid>`

Resultado Esperado: `Id` gerado automaticamente; propriedades refletem os valores atribuídos.

Status:  Passou

CT-013 — Orcamento é instanciado com DotacaoInicial e DotacaoAtualizada

Arquivo: `Entities/OrcamentoTests.cs`

Dados de Entrada: `Ano=2026, DotacaoInicial=450000, DotacaoAtualizada=500000, OrgaoGovernoId=<Guid>`

Resultado Esperado: Valores decimais armazenados com precisão; `Id` não vazio.

Status:  Passou

Módulo: Services — DashboardService

CT-014 — GetResumoAsync retorna KPIs com PercentualExecutado calculado

Arquivo: `Services/DashboardServiceTests.cs`

Pré-condições: Mock de `IDashboardQueryService` configurado com `TotalEmpenhado=1.000.000, TotalPago=600.000`.

Passos:

1. Chamar `_sut.GetResumoAsync()`.
2. Verificar os campos do DTO retornado.

Resultado Esperado: `PercentualExecutado` = 60 (600k / 1M × 100). Verifica que o método de query foi chamado exatamente uma vez.

Status:  Passou

CT-015 — GetResumoAsync retorna 0% quando não há empenhos

Arquivo: `Services/DashboardServiceTests.cs`

Pré-condições: Mock retorna `TotalEmpenhado=0`.

Resultado Esperado: `PercentualExecutado` = 0 — divisão por zero tratada corretamente.

Status:  Passou

CT-016 — GetResumoAsync filtra por ano quando informado

Arquivo: `Services/DashboardServiceTests.cs`

Pré-condições: Mock de query configurado para `ano=2025`.

Passos:

1. Chamar `_sut.GetResumoAsync(2025)`.

Resultado Esperado: A query é chamada com `ano=2025`; o DTO contém o total esperado para o ano.

Status:  Passou

CT-017 — GetComparativoOrgaosAsync retorna lista de órgãos

Arquivo: `Services/DashboardServiceTests.cs`

Pré-condições: Mock retorna 2 órgãos para 2025.

Resultado Esperado: DTO com `Ano=2025` e 2 itens em `Orgaos`.

Status:  Passou

CT-018 — GetDrillDownAsync retorna dados hierárquicos por órgão

Arquivo: `Services/DashboardServiceTests.cs`

Pré-condições: Mock retorna 1 item de drill-down para órgão "001".

Resultado Esperado: DTO com `CodigoOrgao="001"` e 1 item em `Itens`.

Status:  Passou

Módulo: Services — PesquisaService

CT-019 — PesquisaGlobalAsync busca por CNPJ numérico (14 dígitos)

Arquivo: `Services/PesquisaServiceTests.cs`

Pré-condições: Mock de `IContratoRepository.SearchByCnpjAsync` configurado.

Dados de Entrada: `cnpj = "11222333000181"`

Resultado Esperado: `SearchByCnpjAsync` é chamado com o CNPJ limpo; `TotalResultados > 0`.

Status:  Passou

CT-020 — PesquisaGlobalAsync detecta CNPJ formatado com pontuação

Arquivo: `Services/PesquisaServiceTests.cs`

Dados de Entrada: `"11.222.333/0001-81"` (formatado)

Passos:

1. Chamar `PesquisaGlobalAsync("11.222.333/0001-81")`.
2. Verificar que `SearchByCnpjAsync` foi chamado com `"11222333000181"`.
3. Verificar que `SearchByFornecedorAsync` **não** foi chamado.

Resultado Esperado: O serviço sanitiza o CNPJ e usa o fluxo de busca por CNPJ.

Status:  Passou

CT-021 — PesquisaGlobalAsync busca por nome quando termo não é CNPJ

Arquivo: `Services/PesquisaServiceTests.cs`

Dados de Entrada: `"Empresa ABC"` (texto livre)

Resultado Esperado: `SearchByFornecedorAsync("Empresa ABC")` chamado uma vez.

Status:  Passou

CT-022 — PesquisaGlobalAsync lança ArgumentException para termo vazio

Arquivo: `Services/PesquisaServiceTests.cs`

Dados de Entrada: `""`, `null`, `" "`

Resultado Esperado: `ArgumentException` lançada antes de qualquer chamada ao repositório.

Status:  Passou

CT-023 — ExportarCsvAsync retorna bytes com dados do empenho

Arquivo: `Services/PesquisaServiceTests.cs`

Pré-condições: Mock retorna 1 empenho.

Resultado Esperado: CSV contém `"EMP-001"` e `"Empresa A"`; `result.Length > 0`.

Status:  Passou

CT-024 — ExportarCsvAsync retorna apenas o cabeçalho quando não há dados

Arquivo: `Services/PesquisaServiceTests.cs`

Pré-condições: Mock retorna coleção vazia.

Resultado Esperado: CSV gerado contém apenas a linha de cabeçalho ("`NumeroEmpenho; ...`"); sem linhas de dados.

Status:  Passou

CT-025 — ExportarCsvAsync aplica filtro de ano quando informado

Arquivo: `Services/PesquisaServiceTests.cs`

Dados de Entrada: `ano=2025`

Resultado Esperado: `FindAsync` é chamado exatamente uma vez com a expressão de filtro (verificação de invocação).

Status:  Passou

Módulo: Services — DataSyncService

CT-026 — SyncEmpenhosAsync insere empenho novo

Arquivo: `Services/DataSyncServiceTests.cs`

Pré-condições: 1 órgão cadastrado; API externa retorna 1 empenho novo (não existe no banco).

Resultado Esperado: `AddAsync` chamado 1 vez; `CommitAsync` chamado 1 vez; retorna `count=1`.

Status:  Passou

CT-027 — SyncEmpenhosAsync atualiza empenho existente (upsert)

Arquivo: `Services/DataSyncServiceTests.cs`

Pré-condições: 1 órgão; empenho já existe no banco com `Valor=30.000`; API retorna `Valor=50.000`.

Resultado Esperado: `AddAsync` não é chamado; `Valor` do existente atualizado para `50.000`.

Status:  Passou

CT-028 — SyncEmpenhosAsync sanitiza CNPJ ao inserir empenho

Arquivo: `Services/DataSyncServiceTests.cs`

Pré-condições: API retorna CNPJ "`11.222.333/0001-81`" com pontuação.

Resultado Esperado: Empenho inserido com `CnpjCredor = "11222333000181"` (apenas dígitos).

Status:  Passou

CT-029 — SyncEmpenhosAsync retorna zero quando não há órgãos cadastrados

Arquivo: `Services/DataSyncServiceTests.cs`

Pré-condições: Repositório de órgãos retorna lista vazia.

Resultado Esperado: Retorna `0`; API externa **não** é consultada; `CommitAsync` **não** é chamado.

Status:  Passou

CT-030 — SyncEmpenhosAsync retorna zero quando API externa retorna lista vazia

Arquivo: `Services/DataSyncServiceTests.cs`

Pré-condições: 1 órgão cadastrado; API retorna `[]`.

Resultado Esperado: Retorna `0`; `AddAsync` **não** chamado; `CommitAsync` chamado (mesmo sem itens).

Status:  Passou

CT-031 — SyncContratosAsync insere contrato novo com CNPJ sanitizado

Arquivo: `Services/DataSyncServiceTests.cs`

Pré-condições: API retorna 1 contrato novo; repositório retorna `null` para o número do contrato.

Resultado Esperado: `AddAsync` chamado com CNPJ `"22333444000155"` (sanitizado); `CommitAsync` chamado.

Status:  Passou

CT-032 — SyncAllAsync retorna resultado combinado de empenhos e contratos

Arquivo: `Services/DataSyncServiceTests.cs`

Pré-condições: 1 empenho novo + 1 contrato novo nas APIs externas.

Resultado Esperado: `result.EmpenhosProcessados = 1`; `result.ContratosProcessados = 1`.

Status:  Passou

CT-033 — SyncAllAsync retorna SyncedAt com timestamp recente

Arquivo: `Services/DataSyncServiceTests.cs`

Pré-condições: APIs externas retornam listas vazias.

Resultado Esperado: `result.SyncedAt ≤ DateTime.UtcNow`.

Status:  Passou

Módulo: Controllers — DashboardController

CT-034 — GET /dashboard/resumo retorna HTTP 200 com KPIs

Arquivo: `Controllers/DashboardControllerTests.cs`

Pré-condições: Mock de `IDashboardService` configurado com DTO válido.

Resultado Esperado: `OkObjectResult` com `DashboardResumoDto` contendo `TotalEmpenhado = 1.000.000`.

Status:  Passou

CT-035 — GET /dashboard/resumo aplica filtro de ano corretamente

Arquivo: `Controllers/DashboardControllerTests.cs`

Pré-condições: Mock configurado para `ano=2025`.

Passos:

1. Chamar `GetResumo(2025)`.
2. Verificar que o serviço foi chamado com `ano=2025`.

Resultado Esperado: `GetResumoAsync(2025)` chamado exatamente uma vez; retorna DTO correto.

Status:  Passou

CT-036 — GET /dashboard/comparativo retorna HTTP 200 com lista de órgãos

Arquivo: `Controllers/DashboardControllerTests.cs`

Pré-condições: Mock retorna `ComparativoOrgaosDto` com `Ano=2025` e 1 órgão.

Resultado Esperado: `OkObjectResult` com `Ano=2025`.

Status:  Passou

CT-037 — GET /dashboard/evolucao retorna HTTP 200 com dados de drill-down

Arquivo: `Controllers/DashboardControllerTests.cs`

Pré-condições: Mock retorna `DrillDownDto` para órgão "001".

Resultado Esperado: `OkObjectResult` com `CodigoOrgao = "001"`.

Status:  Passou

CT-038 — GET /dashboard/evolucao aplica filtro de ano quando informado

Arquivo: `Controllers/DashboardControllerTests.cs`

Dados de Entrada: `codigoOrgao="002", ano=2025`

Resultado Esperado: Serviço chamado com `("002", 2025)`; DTO retornado com `CodigoOrgao="002"`.

Status:  Passou

Módulo: Controllers — PesquisaController

CT-039 — GET /pesquisa/global retorna HTTP 200 com resultados

Arquivo: `Controllers/PesquisaControllerTests.cs`

Pré-condições: Mock retorna `PesquisaResultDto` com 3 resultados.

Resultado Esperado: `OkObjectResult` com `TermoBuscado = "Empresa A"`.

Status:  Passou

CT-040 — GET /pesquisa/global retorna HTTP 400 quando termo é vazio

Arquivo: `Controllers/PesquisaControllerTests.cs`

Pré-condições: Mock lança `ArgumentException` para termo vazio.

Resultado Esperado: `BadRequestObjectResult`.

Status:  Passou

CT-041 — GET /exportar/csv retorna arquivo CSV com content-type correto

Arquivo: `Controllers/PesquisaControllerTests.cs`

Pré-condições: Mock retorna bytes CSV.

Resultado Esperado: `FileContentResult` com `ContentType = "text/csv"` e nome de arquivo contendo `".csv"`.

Status:  Passou

Módulo: ExternalClients — TcePEDataClient

CT-042 — GetReceitasAsync desserializa o formato JSON do TCE corretamente

Arquivo: `ExternalClients/TcePEDataClientTests.cs`

Pré-condições: `HttpMessageHandler` mockado retorna JSON no formato envelope `{ resposta: { conteudo: [...] } }`.

Resultado Esperado: Lista com 1 item; `ValorReceita = 150.000`; `Origem = "Imposto"`.

Status:  Passou

CT-043 — GetReceitasAsync lança `HttpRequestException` quando API retorna erro 500

Arquivo: `ExternalClients/TcePEDataClientTests.cs`

Pré-condições: Handler mockado retorna `HttpStatusCode.InternalServerError`.

Resultado Esperado: `HttpRequestException` lançada (uso de `EnsureSuccessStatusCode`).

Status:  Passou

CT-044 — GetEmpenhosByOrgaoAsync desserializa empenhos corretamente

Arquivo: `ExternalClients/TcePEDataClientTests.cs`

Pré-condições: Handler retorna JSON com 1 empenho.

Resultado Esperado: `NumeroEmpenho = "EMP-001"`, `Valor = 50.000`, `NaturezaDespesa = "3.3.90.30"`.

Status:  Passou

CT-045 — GetEmpenhosByOrgaoAsync retorna lista vazia quando API retorna erro 404

Arquivo: `ExternalClients/TcePEDataClientTests.cs`

Pré-condições: Handler retorna `HttpStatusCode.NotFound`.

Resultado Esperado: Lista vazia (sem lançar exceção — verificação manual de `IsSuccessStatusCode`).

Status:  Passou

CT-046 — GetContratosAsync desserializa contratos corretamente

Arquivo: `ExternalClients/TcePEDataClientTests.cs`

Pré-condições: Handler retorna JSON com 1 contrato.

Resultado Esperado: `NumeroContrato = "CT-2025-001", ValorContrato = 200.000`.

Status:  Passou

CT-047 — GetContratosAsync retorna lista vazia quando API está indisponível

Arquivo: `ExternalClients/TcePEDataClientTests.cs`

Pré-condições: Handler retorna `HttpStatusCode.ServiceUnavailable`.

Resultado Esperado: Lista vazia retornada sem lançar exceção.

Status:  Passou

CT-048 — GetOrcamentoAsync retorna lista vazia quando API retorna erro

Arquivo: `ExternalClients/TcePEDataClientTests.cs`

Pré-condições: Handler retorna `HttpStatusCode.BadGateway`.

Resultado Esperado: Lista vazia retornada sem lançar exceção.

Status:  Passou

Módulo: Middlewares — GlobalExceptionHandlerMiddleware

Componente cross-cutting que captura exceções não tratadas e retorna respostas padronizadas.

CT-049 — Middleware passa para o próximo handler quando não há exceção

Arquivo: `Middlewares/GlobalExceptionHandlerMiddlewareTests.cs`

Pré-condições: `RequestDelegate` mockado que apenas seta uma flag.

Passos:

1. Invocar `middleware.InvokeAsync(context)`.
2. Verificar que o delegate seguinte foi chamado.

Resultado Esperado: `nextCalled = true`; `StatusCode` permanece 200.

Status:  Passou

CT-050 — Middleware retorna HTTP 400 para `ArgumentException`

Arquivo: `Middlewares/GlobalExceptionMiddlewareTests.cs`

Pré-condições: `RequestDelegate` lança `ArgumentException("Termo de busca inválido.")`.

Resultado Esperado: `StatusCode = 400`; corpo contém a mensagem da exceção.

Status:  Passou

CT-051 — Middleware retorna HTTP 404 para `NotFoundException`

Arquivo: `Middlewares/GlobalExceptionMiddlewareTests.cs`

Pré-condições: `RequestDelegate` lança `NotFoundException("Recurso não encontrado.")`.

Resultado Esperado: `StatusCode = 404`; `DomainException.StatusCode` é respeitado.

Status:  Passou

CT-052 — Middleware retorna HTTP 500 para exceções não tratadas

Arquivo: `Middlewares/GlobalExceptionMiddlewareTests.cs`

Pré-condições: `RequestDelegate` lança `InvalidOperationException`.

Resultado Esperado: `StatusCode = 500`; corpo contém `"internal server error"` (mensagem genérica, sem vaziar detalhes internos).

Status:  Passou

CT-053 — Middleware retorna JSON estruturado com `statusCode`, `message` e `timestamp`

Arquivo: `Middlewares/GlobalExceptionMiddlewareTests.cs`

Pré-condições: `RequestDelegate` lança `ArgumentException`.

Passos:

1. Invocar o middleware.
2. Verificar `ContentType = "application/json"`.
3. Parsear o corpo como JSON.
4. Verificar presença das propriedades `statusCode`, `message` e `timestamp`.

Resultado Esperado: Corpo JSON válido com as três propriedades obrigatórias.

Status:  Passou

CT-054 — Middleware respeita `StatusCode` da `DomainException` derivada

Arquivo: `Middlewares/GlobalExceptionMiddlewareTests.cs`

Pré-condições: `NotFoundException` herda `DomainException` com `StatusCode=404`.

Resultado Esperado: context.Response.StatusCode = 404.

Status:  Passou

Resumo de Cobertura

Camada	Componentes Testados
Domain — Entities	OrgaoGoverno, Receita, Orcamento
Application — Helpers	CnpjHelper, McaspMapper
Application — Services	DashboardService, PesquisaService, DataSyncService
Infrastructure — ExternalClients	TcePEDataClient
API — Controllers	DashboardController, PesquisaController
API — Middlewares	GlobalExceptionMiddleware

Nota: Os testes unitários utilizam mocks (Moq) para isolar dependências externas (banco de dados, APIs HTTP, logger). Entidades de domínio são testadas diretamente, sem mocks.